

Ottimizzazione della Memoria nella Programmazione Dinamica per l'Allineamento di Sequenze

Iacopo Giansanti

September 2026

Indice

1	Introduzione	2
2	Definizione del Global Alignment problem e dei prerequisiti essenziali	2
2.1	Rappresentazione delle sequenze biologiche	2
2.2	Allineamento e modello di scoring	3
2.3	Programmazione dinamica	5
2.4	Longest Common Subsequence	5
2.5	Alignment graph e Manhattan Tourist Problem	5
2.6	Global Alignment Problem	5
3	Algoritmo di Needleman-Wunsch	5
3.1	Ricorrenza, inizializzazione e traceback	5
3.2	Pseudocodice	5
3.3	Analisi della complessità	5
3.4	Implementazione	5
4	Algoritmo di Hirschberg	5
4.1	Ricorrenza, inizializzazione e traceback	5
4.2	Pseudocodice	5
4.3	Analisi della complessità	5
4.4	Implementazione	5
5	Metodologia sperimentale	5
5.1	Dataset	5
5.2	Ambiente hardware e software	5
5.3	Misura dei tempi e della memoria	5

1 Introduzione

2 Definizione del Global Alignment problem e dei prerequisiti essenziali

2.1 Rappresentazione delle sequenze biologiche

Il DNA è costituito da quattro tipi di nucleotidi, caratterizzati dalle basi seguenti azotate:

- Adenina, A ;
- Citosina, C ;
- Guanina, G ;
- Timina, T .

I due filamenti della molecola sono complementari e antiparalleli, con appaiamenti

$$A \longleftrightarrow T \quad , \quad C \longleftrightarrow G$$

Dal punto di vista computazione, un singolo filamento può essere rappresentato come una stringa sull'alfabeto con orientamenti opposti.

$$\Sigma_{\text{DNA}} = \{A, C, G, T\}.$$

Quindi una stringa di lunghezza n ha forma

$$x = x_1 x_2 \cdots x_n, \quad x_i \in \Sigma_{\text{DNA}} \quad \forall i \in \{1, \dots, n\}.$$

Una rappresentazione computazionale può allora considerare la stringa di DNA e i due filamenti tra di loro complementari.

Una *substring* è una porzione contigua di una stringa. Data una stringa di lunghezza n esiste una substring di lunghezza k per $k \leq n$.

Un *k-mero* è una substring di lunghezza k , $\exists k \in \mathbb{Z}_{\geq 0}$

Una *subsequence* è invece una nuova stringa ottenuta eliminando 0 o più elementi della stringa originale mantenendone l'ordine relativa.

Mostriamo un esempio: sia s una stringa che rappresenta una sequenza genomica,

$$s := \text{ACCT},$$

allora AC può essere sia una substring che una subsequence, mentre AT è definibile soltanto come subsequence.

Due sequenze genomiche omologhe possono differire tra loro a causa di mutazioni. Consideriamo tre eventi possibili associati a tali mutazioni:

$$\begin{array}{lll} \text{sostituzione :} & s = \text{AC} & \longrightarrow s' = \text{AA} \\ \text{inserzione :} & s = \text{AC} & \longrightarrow s' = \text{AAC} \\ \text{delezione :} & s = \text{AAC} & \longrightarrow s' = \text{AC} \end{array}$$

$$s = AC \xrightarrow{\text{inserzione}} AAC$$

Gli ultimi due eventi verranno ogni tanto trattati come appartenenti a un unico tipo, dal nome *indel*.

Supponiamo, quindi, di avere due sequenze genomiche omologhe che differiscono a causa di mutazioni genetiche; come possiamo quantificare e individuare la somiglianza tra le due sequenze?

2.2 Allineamento e modello di scoring

Stiamo quindi iniziando a definire il problema di nostro interesse.

Prima però chiariamo velocemente l'idea di problema e algoritmo in generale.

Dato che un nostro obiettivo è quello di misurare la differenza tra due sequenze genetiche, trattiamo adesso la *distanza di Hamming*, che è lo strumento più semplice per valutare la somiglianza tra due stringhe. Siano x, y due stringhe della stessa lunghezza, allora la distanza di Hamming è

$$d_H(x, y) := |\{i : x_i \neq y_i\}|,$$

ovvero il numero di posizioni in cui due stringhe differiscono. La distanza di Hamming è però applicabile soltanto a stringhe della stessa lunghezza e non modella adeguatamente inserzioni e delezioni, che possono inoltre alterare la corrispondenza tra le posizioni delle due sequenze.

Abbiamo quindi bisogno di una misura più precisa. Definiamo allora l'*edit distance*, o anche distanza di Levenshtein, $d(v, w)$, come il numero minimo di operazioni di editing (mutazioni) necessarie per trasformare una sequenza v in una sequenza w .

L'edit distance fornisce una misura della dissimilarità tra due sequenze. Per rappresentare esplicitamente le corrispondenze tra i loro simboli introduciamo ora il concetto di **allineamento**.

Siano v e w due sequenze. Un allineamento di v e w è una matrice

$$\mathcal{A} \in (\Sigma_{\text{DNA}} \cup \{-\})^{2 \times L},$$

con,

$$\max\{|v|, |w|\} \leq L \leq |v| + |w|,$$

tale che, eliminando i simboli di gap dalla prima riga, si ottiene v , mentre eliminandoli dalla seconda si ottiene w . Non sono ammesse colonne contenenti due gap.

Ogni colonna di un allineamento corrisponde a uno dei seguenti casi:

- match;
- mismatch (sostituzione);
- inserzione;
- delezione;

Visualizziamo cosa è stato appena detto:

siano v_{aligned} e w_{aligned} le sequenze allineate e $\mathcal{A}$ il loro allineamento, allora,

$$\begin{array}{rcl} v_{\text{aligned}} & : & A \ C \ T \ - \ G \ C \ A \\ w_{\text{aligned}} & : & A \ - \ T \ C \ G \ C \ A \end{array} = \mathcal{A}$$

Per valutare la similarità tra simboli introduciamo una funzione di **scoring**,

$$\delta : ((\Sigma_{\text{DNA}} \cup \{-\})^2 \setminus \{(-, -)\}) \rightarrow \mathbb{R},$$

che date due basi a e b restituisce un valore reale, $\delta(a, b)$.

Una funzione di scoring (anche detta modello) per una sequenza di DNA è generalmente definita attribuendo un premio in caso di match e una penalità in caso contrario

$$\delta(a, b) = \begin{cases} +\mu, & a = b, \\ -\eta, & a \neq b, \\ -\sigma, & \text{se esattamente uno tra } a, b \text{ è un gap} \end{cases}$$

con $\mu, \eta, \sigma > 0$. Il modello in cui mismatch e gap hanno la stessa penalità si ottiene come caso particolare ponendo $\eta = \sigma$.

Grazie alla funzione di scoring possiamo anche definire la **scoring matrix**,

δ	A	C	G	T	$-$
A	$+\mu$	$-\eta$	$-\eta$	$-\eta$	$-\sigma$
C	$-\eta$	$+\mu$	$-\eta$	$-\eta$	$-\sigma$
G	$-\eta$	$-\eta$	$+\mu$	$-\eta$	$-\sigma$
T	$-\eta$	$-\eta$	$-\eta$	$+\mu$	$-\sigma$
$-$	$-\sigma$	$-\sigma$	$-\sigma$	$-\sigma$	n.a.

Sia

$$\mathcal{A} = \begin{pmatrix} v_1 & \dots & v_L \\ w_1 & \dots & w_L \end{pmatrix},$$

allora, definiamo lo score complessivo dell'allineamento come

$$\text{score}(\mathcal{A}) = \sum_{i=1}^L \delta(v_i, w_i).$$

Nel modello precedente ogni gap viene penalizzato indipendentemente. Una successione di gap consecutivi può tuttavia essere interpretata come risultato di un unico evento di inserzione o delezione. Per questo motivo vengono spesso utilizzate penalità affini, che distinguono il costo di apertura di un gap da quello della sua estensione. Tale generalizzazione richiede stati aggiuntivi nella programmazione dinamica; nel seguito ci concentreremo tuttavia sul modello a penalità lineare utilizzato dagli algoritmi oggetto del confronto.

Un problema deve specificare, almeno, input e output. Mentre un algoritmo ha il compito di restituire a ogni istanza valida dell'input, un output corretto.

Delineamo gli obiettivi computazionali che ci guideranno nel percorso:

- calcolare il valore ottimo;
- ricostruire una soluzione ottima;
- riuscire a farlo utilizzando poca memoria.

Introduciamo allora tre stati per modellare la situazione appena trattata:

- (1) $S_{i,j}$: miglior allineamento dei prefissi che termina normalmente;
- (2) $S_{i,j}^\downarrow$: miglior allineamento che termina con una delezione, quindi con un gap nella seconda sequenza;
- (3) $S_{i,j}^\rightarrow$: miglior allineamento che termina con una inserzione, quindi con un gap nella prima sequenza.

Le transizioni dei due stati di gap, (1) e (2), distinguono apertura e continuazione:

$$S_{i,j}^\downarrow = \max \left\{ S_{i-1,j}^\downarrow - \sigma, S_{i-1,j} - (\rho + \sigma) \right\},$$

$$S_{i,j}^{\rightarrow} = \max \{ S_{i,j-1}^{\rightarrow} - \sigma, S_{i,j-1} - (\rho + \sigma) \}.$$

Lo stato principale (3) combina quindi il match/mismatch con i due stati di gap:

$$S_{i,j} = \max \{ S_{i-1,j-1} + \delta(v_i, w_j), S_{i,j}^{\downarrow}, S_{i,j}^{\rightarrow} \}.$$

Avendo definito il sistema di valutazione e di allineamento, possiamo studiarne la complessità

2.3 Programmazione dinamica

2.4 Longest Common Subsequence

2.5 Alignment graph e Manhattan Tourist Problem

2.6 Global Alignment Problem

3 Algoritmo di Needleman-Wunsch

3.1 Ricorrenza, inizializzazione e traceback

3.2 Pseudocodice

3.3 Analisi della complessità

3.4 Implementazione

4 Algoritmo di Hirschberg

4.1 Ricorrenza, inizializzazione e traceback

4.2 Pseudocodice

4.3 Analisi della complessità

4.4 Implementazione

5 Metodologia sperimentale

5.1 Dataset

5.2 Ambiente hardware e software

5.3 Misura dei tempi e della memoria